



React Hooks for SPFx: Build Cleaner, Faster, Better

Don Kirkham - Microsoft MVP, M365 Development & Microsoft Graph



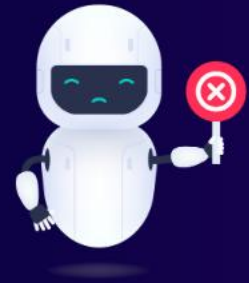
PHOTOGRAPHY DURING THE SESSION

Feel free to capture the moment! Taking pictures of the presentation, during the session is perfectly fine.



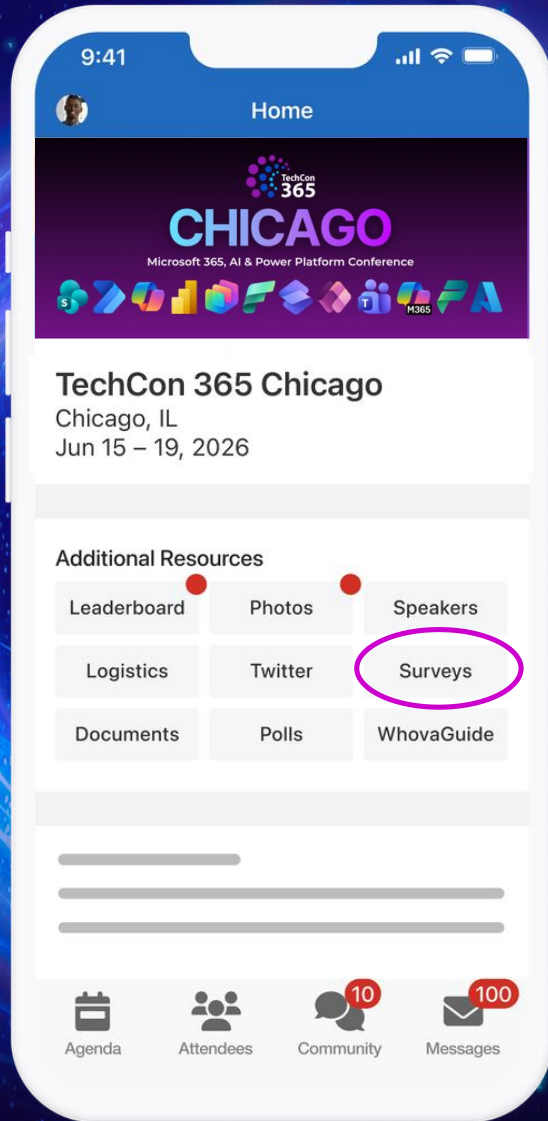
SELFIES WITH THE SPEAKER

We encourage interaction! If you'd like to take a selfie with the speaker, don't hesitate to ask. Most speakers will be happy to oblige during appropriate breaks.



RECORDING AND LIVE STREAMING

Recording or live streaming the session, in part or in full, is strictly prohibited. Thank you for respecting our content and Speakers.



How was the session?

Search for *Ukhova* in the
App Store or Google Play



Fill out the Session Surveys in the
TechCon 365 Chicago Event App
and be eligible to **WIN PRIZES!**

DON KIRKHAM

MICROSOFT MVP, M365 DEVELOPMENT & MICROSOFT GRAPH



Arlington, Texas, USA



Senior Technology Advisor



<https://donkirkham.com>



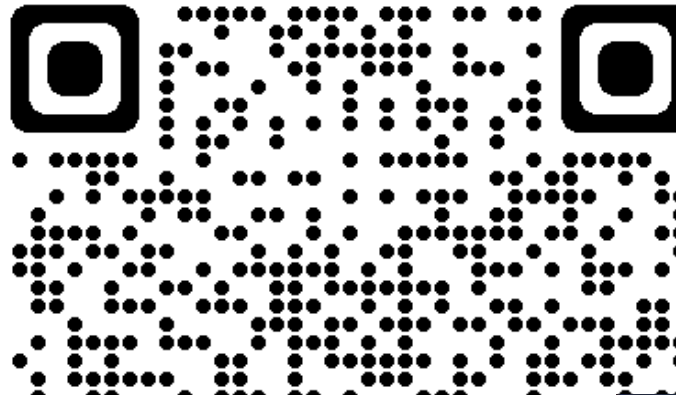
@DonKirkham

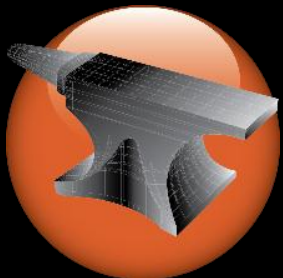


/in/DonKirkham



DonKirkham





SOLUTIONFOUNDRY®

M365 Solutions and Beyond. Built for your needs. By our experts.

Secure Microsoft 365 modernization, automation, and custom development—delivered by our team of senior consultants, developers, and Microsoft MVPs

Core Microsoft 365 services and custom tools

- SPFx Development – SharePoint Framework solutions that extend what M365 can do out of the box.
- SharePoint Modernization – Plan, migrate and transition from Classic SharePoint to the Modern experience.
- Power Platform– Data Visualization with Power BI, workflow automation with Power Automate, and user-friendly Power Apps.
- AI & Copilot Integration – Copilot and other AI tools design and implementation.

Federal & Regulated Program Support

- Decades of experience supporting federal and highly regulated organizations with compliance-driven systems and mission support solutions.
- Primary architects and implementers of enterprise portals, dashboards, and knowledge platforms that standardize processes, improve visibility, and streamline reporting.

General Qualifications

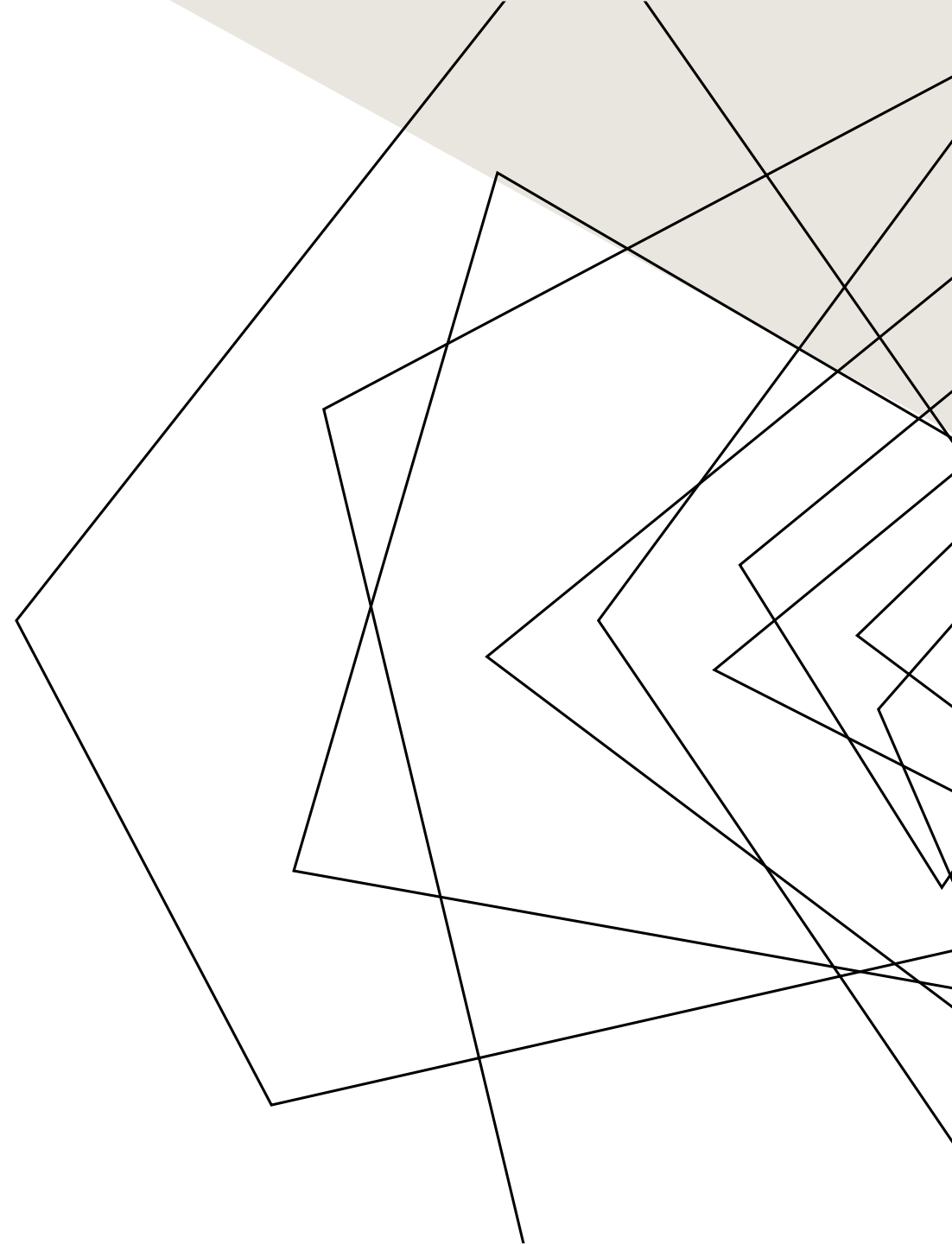
- 3 decades of federal contracting experience
- Microsoft MVP expertise and industry recognition
- Proven ability to embed with customer teams as trusted advisors
- Deep understanding of federal compliance, security, and accountability requirements
- Vast experience with business process, data management and communications analysis, improvement and automation.

Representative Solution Types

- Enterprise portals, dashboards, and reporting for operational visibility
- Compliance workflows, document management, and audit readiness solutions
- Knowledge platforms and self-service support content organized by mission area

AGENDA

- What are Function(al) Components and Hooks
- Classes vs Function Components
- Converting your SPFx project
- Hooks, Hooks, Hooks!



FUNCTIONAL COMPONENTS + HOOKS (FC+HOOKS)

THE HISTORY

- 2015 – FCs introduced as Class alternative
 - Stateless
- 2019 – React Hooks introduced in React 16.8
 - Allowed granular state management in FC
- SharePoint Framework (SPFx)
 - Microsoft Yeoman Generator
 - Default scaffolds with Class Components
 - FC+Hooks only available in SPFx 1.9+
- Function vs Functional
 - Functional confusing based on Functional Computing
 - Functional Component is a defined React element
 - Function results in cleaner code
 - Functional uses implicit properties (inconsistent)
 - “Preferred” way to do function-based components

CLASS COMPONENT (DEFAULT SPFX)

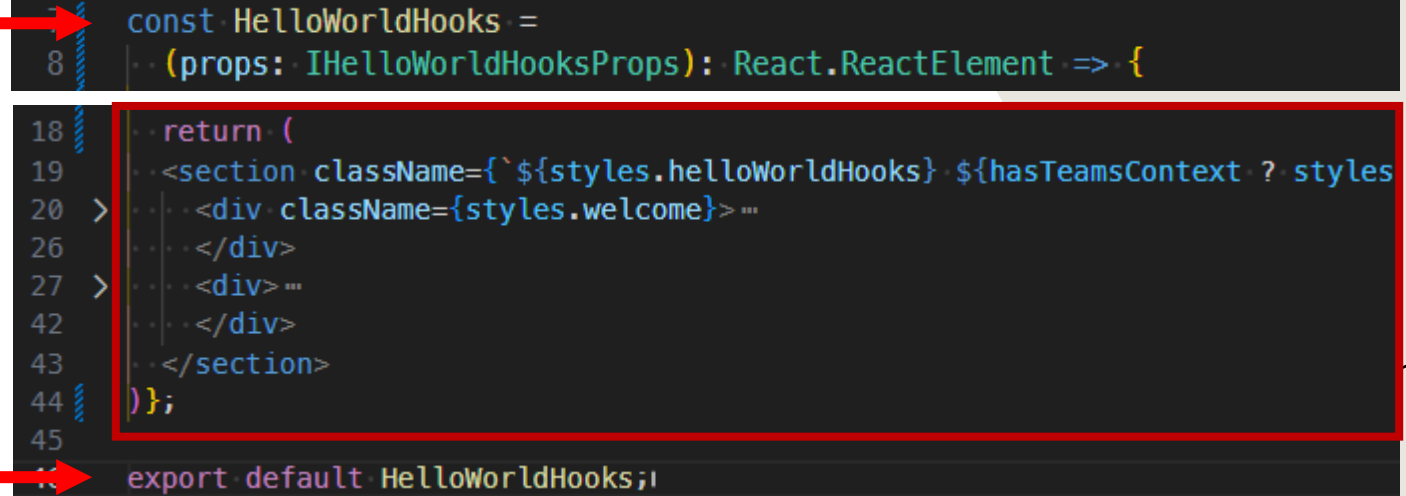
```
1 > import * as React from 'react'; ...
3
4 > export interface ILifecycleClassicProps { ...
10 }
11
12 > export interface ILifecycleClassicState { ...
16 }
17
18 > export default class LifecycleClassic extends React.Component<ILifecycleClassicProps, ILifecycleClassicState> {
19
20   protected timer?: number;
21
22   // Constructor: initialize state and bind handlers
23 >   constructor(props: ILifecycleClassicProps) { ...
27   }
28
29   // componentDidMount: log mount event
30 >   componentDidMount(): void { ...
32   }
33
34   // componentDidUpdate: log updates and handle count logic
35 >   componentDidUpdate(prevProps: ILifecycleClassicProps, prevState: ILifecycleClassicState): void { ...
47   }
48
49   // componentWillUnmount: log unmount and clean up
50 >   componentWillUnmount(): void { ...
53   }
54
55 >   // #region other functions ...
99
100 >   render(): JSX.Element { ...
122   }
123 }
```

The diagram illustrates the structure of a Class Component in SPFX. It shows the following components:

- Interfaces:** `ILifecycleClassicProps` and `ILifecycleClassicState` are defined at the top.
- Class Declaration:** The `LifecycleClassic` class is declared, extending `React.Component` with the specified interfaces.
- Lifecycle Methods:** The class implements several lifecycle methods: `constructor`, `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.
- Render Method:** The `render` method is defined at the bottom, returning a `JSX.Element`.

WHAT ARE FUNCTION COMPONENTS?

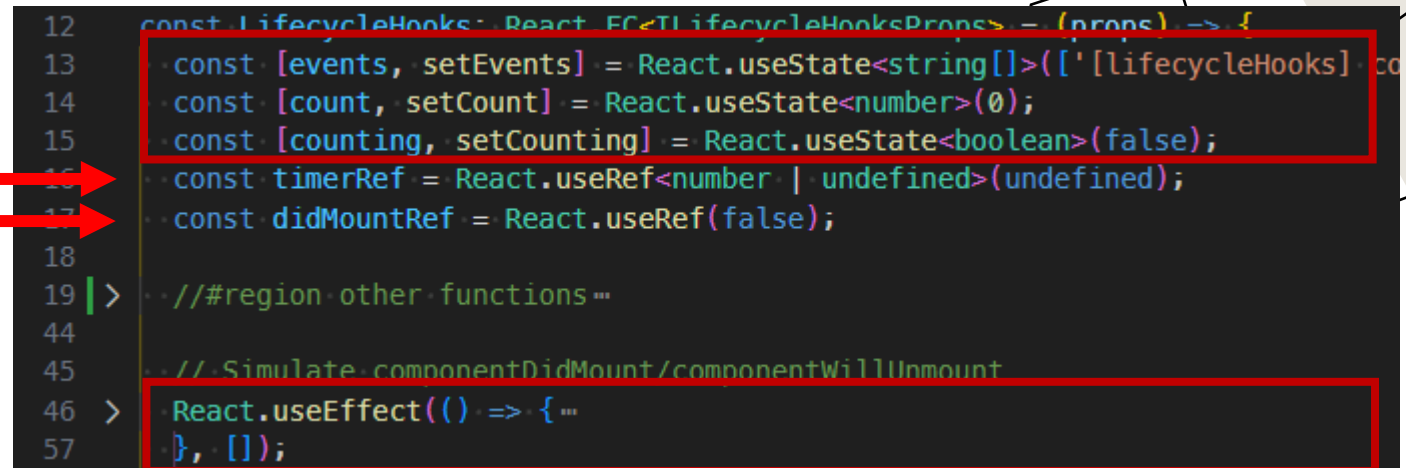
- Simply, JavaScript Functions
 - Accepts Properties
 - Stateless
- Output: React Element (JSX/TSX)
 - Just like render function in class



```
7
8 const HelloWorldHooks =
  (props: IHelloWorldHooksProps): React.ReactElement => {
18
19   return (
20     <section className={` ${styles.helloWorldHooks} ${hasTeamsContext ? styles
26     <div className={styles.welcome}> ...
27     <div> ...
42     </div>
43   </section>
44   );
45
46 export default HelloWorldHooks;
```

WHAT ARE HOOKS?

- Primarily used to manage state
- So much more!



```
12 const LifecycleHooks: React.FC<ILifecycleHooksProps> = (props) => {
13   const [events, setEvents] = React.useState<string[]>([' [lifecycleHooks] co
14   const [count, setCount] = React.useState<number>(0);
15   const [counting, setCounting] = React.useState<boolean>(false);
16   const timerRef = React.useRef<number | undefined>(undefined);
17   const didMountRef = React.useRef(false);
18
19   // #region other functions ...
44
45   // Simulate componentDidMount/componentWillUnmount
46   React.useEffect(() => { ...
47     }, []);
57
```

CONVERSION TO FUNCTION COMPONENTS

- Best done right after new project creation
- No changes in the base webpart
- 2 lines + cleanup
 - declaration
 - remove render
 - add export statement

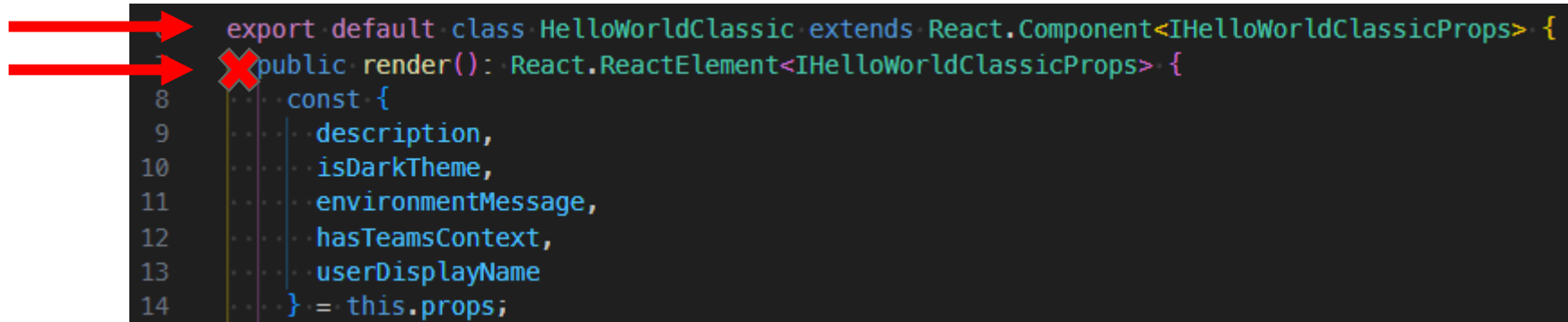


Diagram illustrating the removal of the `render` method from a class component. Two red arrows point to the `export default` and `public render()` lines, which are marked with a red 'X' indicating they are to be removed. The code snippet shows a class `HelloWorldClassic` extending `React.Component` with a `render` method that returns a `const` object containing props like `description`, `isDarkTheme`, `environmentMessage`, `hasTeamsContext`, and `userDisplayName`.

```
export default class HelloWorldClassic extends React.Component<IHelloWorldClassicProps> {  
  public render(): React.ReactElement<IHelloWorldClassicProps> {  
    const {  
      description,  
      isDarkTheme,  
      environmentMessage,  
      hasTeamsContext,  
      userDisplayName  
    } = this.props;  
  }  
}
```

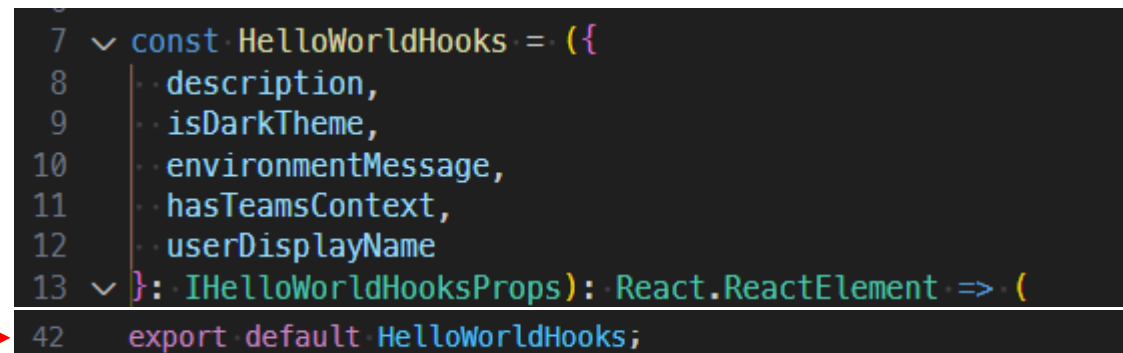


Diagram illustrating the creation of a function component and the final export statement. A red arrow points to the `export default` line at the bottom. The code snippet shows a `const` declaration for `HelloWorldHooks` as a function component, followed by the `export default` statement.

```
const HelloWorldHooks = ({  
  description,  
  isDarkTheme,  
  environmentMessage,  
  hasTeamsContext,  
  userDisplayName  
}: IHelloWorldHooksProps): React.ReactElement => (  
  export default HelloWorldHooks;
```

GAMECHANGERS: useState() & useEffect()

useState()

- replaces this.state, granular state
- no lifecycle methods (useEffects)
- useState declared as a 2 item array
 - first item is variable to hold value
 - second item is function to change the variable value
 - initialization is possible in declaration

```
const [events, setEvents] = React.useState<string[]>(['lifecycle']);  
const [count, setCount] = React.useState<number>(0);  
const [counting, setCounting] = React.useState<boolean>(false);
```

```
    setCount(0);  
    setCounting(false);
```

```
    if (count === 10) {
```

```
      {counting && <div>Counting...</div>}  
    }
```

useEffect()

- function that is called after component renders
- uses 2 parameters
 - 1st parameter is callback function that runs when component renders or updates
 - 2nd parameter is dependency array, which determines when useEffect runs
 - no parameter: always runs
 - []: only runs on first render
 - [value]: runs when value changes
- ¹³ return function can be used to clean up

```
React.useEffect(() => {  
  console.log("Effect runs on every render!");  
}); // No dependency array means it runs every render
```

```
React.useEffect(() => {  
  console.log("Component mounted!");  
}, []); // Empty array means it runs only once
```

```
React.useEffect(() => {  
  console.log(`Count changed to ${count}`);  
}, [count]); // Runs only when `count` updates
```


DEMO TIME!

Just warming up!



useCallback

Preventing unnecessary re-renders in child components.

Particularly useful in cases where you pass functions as props to child components or use them inside event handlers, preventing unnecessary re-renders and improving performance.

Takes two arguments:

- **A function** – The callback you want to memoize.
- **A dependency array** – When dependencies change, the function will be re-created.

```
function Button({ onClick }) {  
  console.log("Button rendered");  
  return <button onClick={onClick}>Click me</button>;  
}  
  
function ParentComponent() {  
  const [count, setCount] = useState(0);  
  
  const increment = useCallback(() => {  
    setCount(prevCount => prevCount + 1);  
  }, []); // Memoized function  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <Button onClick={increment} />  
    </div>  
  );  
}
```

useRef

creates a mutable reference to store values without causing re-renders.

It's commonly used for:

- Accessing & modifying DOM elements (e.g., focusing an input field).
- Persisting values across renders (e.g., tracking previous state or storing intervals).
- Avoiding unnecessary state updates (since useRef doesn't trigger re-renders).

```
function InputFocus() {
  const inputRef = useRef<HTMLInputElement>();

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={() => inputRef.current?.focus()} />
    </div>
  );
}
```

```
return (
  <div style={styles.container}>
    <p>Some content</p>
    <p>More content</p>
    <div ref={inputRef}>
      <input type="text" />
    </div>
  </div>
);
```

```
const [count, setCount] = React.useState<number>(0);
const [counting, setCounting] = React.useState<boolean>(false);
const timerRef = React.useRef<number | undefined>(undefined);
const didMountRef = React.useRef(false);

const startTimer = React.useCallback(() => {
  if (!timerRef.current) {
    timerRef.current = window.setInterval(() => {
      setCount(prev => (prev < 10 ? prev + 1 : prev));
    }, 500);
  }
}, []);

const stopTimer = React.useCallback(() => {
  if (timerRef.current) {
    clearInterval(timerRef.current);
    timerRef.current = undefined;
    setCounting(false);
  }
}, []);

React.useEffect(() => {
  didMountRef.current = true;
  return () => {
    if (timerRef.current) {
      clearInterval(timerRef.current);
      timerRef.current = undefined;
    }
  };
}, []);

React.useEffect(() => {
  if (didMountRef.current) {
    if (count === 10 && counting) {
      stopTimer();
    }
  }
}, [count]);
```

useMemo()

Optimizing performance by memorizing expensive calculations, preventing them from being re-calculated on every render

Takes two arguments:

1. A function – Returns the computed value.
2. A dependency array – The value is only recomputed when dependencies change.

```
function Example() {
  const [count, setCount] = useState(0);

  function SortedList({ items }) {
    const sortedItems = useMemo(() => {
      console.log("Sorting...");
      return [...items].sort();
    }, [items]);

    return <ul>{sortedItems.map((item, index) => <li key={index}>{item}</li>)}</ul>;
  }

  function App() {
    const [items, setItems] = useState(["banana", "apple", "cherry"]);

    function ComputeFactorial({ number }) {
      const factorial = useMemo(() => {
        console.log("Computing factorial...");
        let result = 1;
        for (let i = 2; i <= number; i++) {
          result *= i;
        }
        return result;
      }, [number]);

      return <p>Factorial of {number}: {factorial}</p>;
    }

    return (
      <div>
        <input type="text" value={number} />
        <button value={number} />
        <div>
          <input type="text" value={query} />
          <ul>{filteredItems.map(item => <li key={item}>{item}</li>)}</ul>
        </div>
      </div>
    );
  }
}
```


useContext()

Allows components to access global state without having to pass props manually at every level.

It's useful for managing theme settings, authentication status, user preferences, or any shared state across multiple components.

How It Works

1. Create a Context – Define shared state using `createContext()`.
2. Provide a Context Value – Use `Context.Provider` to pass data down.
3. Consume Context in Components – Use `useContext()` to access the shared state.

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light"); // Default value: "light"

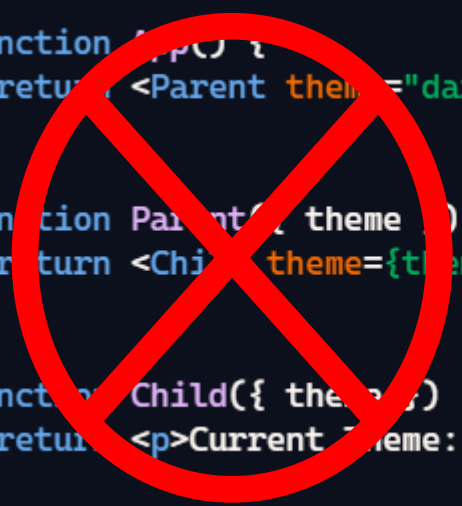
function ChildComponent() {
  const theme = useContext(ThemeContext); // Access context value
  return <p>Current Theme: {theme}</p>;
}

function ParentComponent() {
  return (
    <ThemeContext.Provider value="dark">
      <ChildComponent />
    </ThemeContext.Provider>
  );
}

function App() {
  return (
    <ParentComponent theme="dark" />
  );
}

function Child({ theme }) {
  return <p>Current Theme: {theme}</p>;
}

function App() {
  return (
    <AuthContext.Provider value={{ name: "Alice" }}>
      <Profile />
    </AuthContext.Provider>
  );
}
```



Custom Hooks

Encapsulates reusable logic into functions, making code cleaner and more modular.

They're just JavaScript functions that use built-in hooks like `useState()`, `useEffect()`, or `useContext()` to provide specific functionality.

Why?

- Avoid repeated logic across multiple components
- Improve code readability and organization
- Encapsulate side effects cleanly

```
import { useState, useEffect } from "react";

function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };

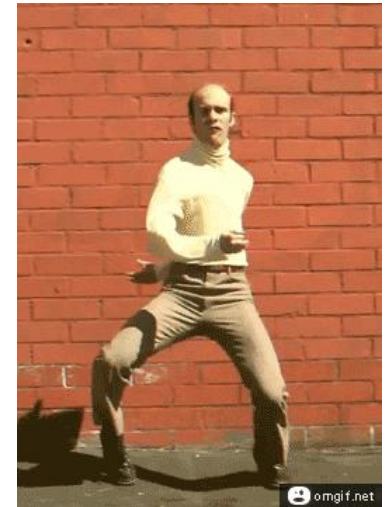
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

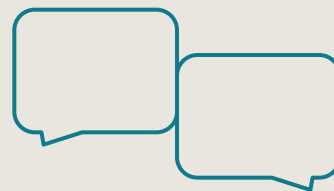
  return size;
}

function Component() {
  const { width, height } = useWindowSize();
  return <p>Window size: {width}x{height}</p>;
}
```

DEMO TIME!

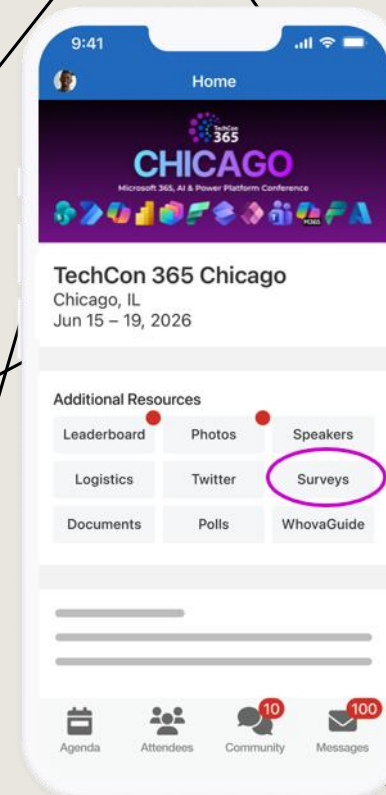
PUT ON YOUR DANCING SHOES AND LET'S HAVE SOME FUN!





Q&A

Questions?



Don Kirkham



<https://donkirkham.com>



@DonKirkham



/in/DonKirkham



github.com/DonKirkham/Presentations

Whova

Please rate my session

- Open the Whova app
- Surveys → fill it out

Be eligible to **win prizes!**